

Лекция 6

Указатели

1 Понятие указателей. Объявление указателей

Указатель – переменная, содержащая адрес объекта. Указатель не несет информации о содержимом объекта, а содержит сведения о том, где размещен объект.

Общая форма объявления указателя:
тип *Имя_Переменной;

Пример:

```
int *a;  
char *ch1, ch2, ch3[10];  
void v;
```

2 Операции над указателями

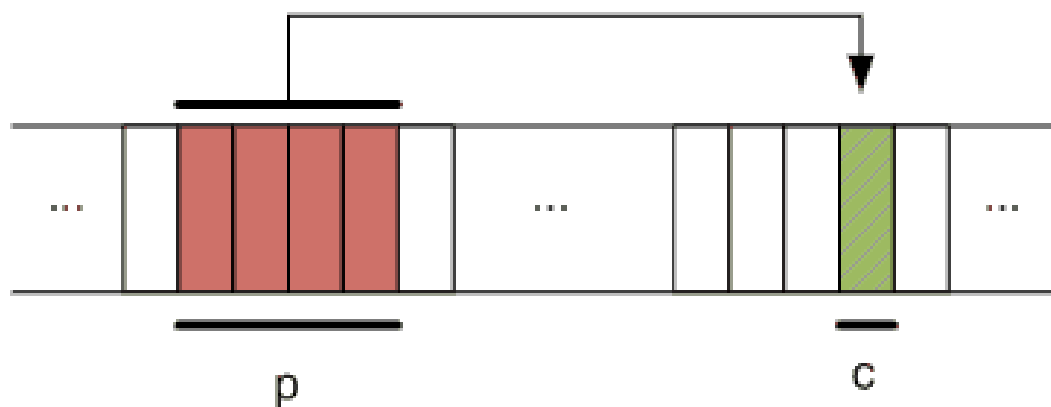
* – получение значение по указанному адресу;

& – определение адреса переменной.

Пример:

```
char c; // переменная
char *p; // указатель
p=&c; // p = адрес c
*p=3;
printf("%d\n", c);
c=10;
printf("%d", *p);
```

3
10



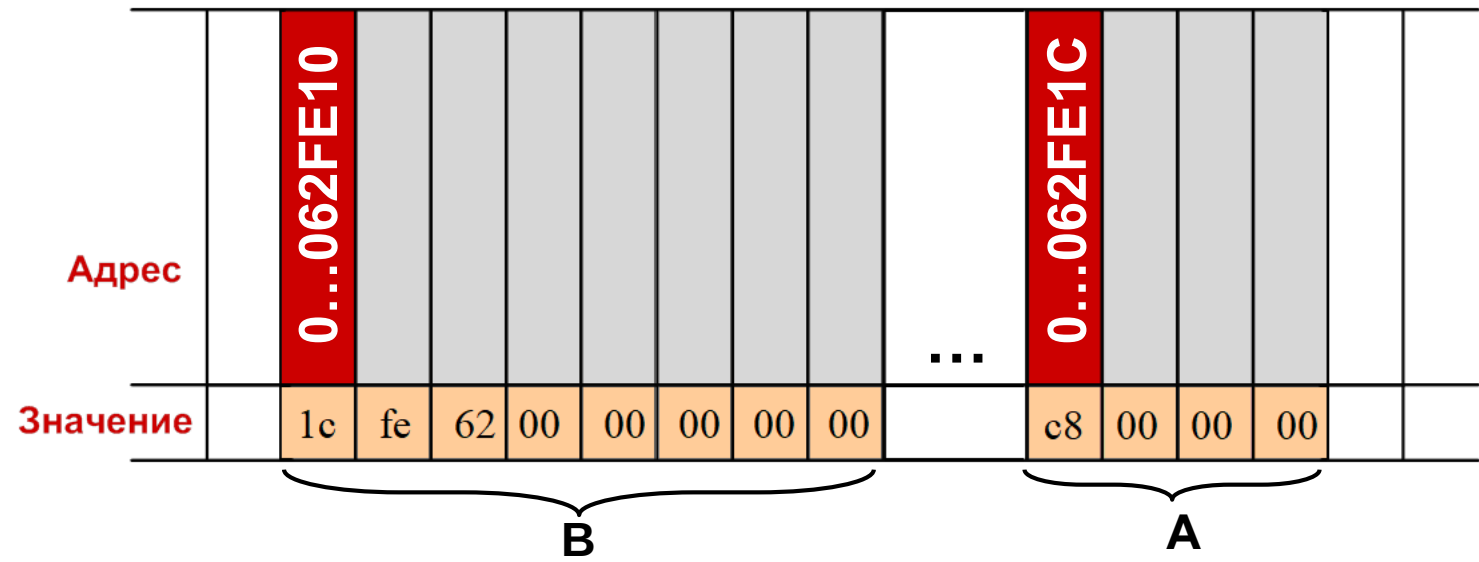
	Переменная	Указатель
Адрес	&c	p
Значение	c	*p

В памяти указатель занимает:

- 4 байта (32-битная версия компилятора);
- 8 байт (64-битная версии компилятора).

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int A, *B;
6     A = 200;
7     B = &A;
8
9     printf("1) Переменная A занимает в памяти %d байта\n", sizeof(A));
10    printf("2) Указатель B занимает в памяти %d байт\n", sizeof(B));
11    printf("3) Значение переменной A равно %d = %x\n", A, A);
12    printf("4) Адрес переменной A равен %p\n", &A);
13    printf("5) Данные по адресу указателя B равны %d = %x\n", *B, *B);
14    printf("6) Значение указателя B равно %x = %p\n", B, B);
15    printf("7) Адрес расположения указателя B равен %p", &B);
16    return 0;
17 }
```

- 1) Переменная A занимает в памяти 4 байта
- 2) Указатель B занимает в памяти 8 байт
- 3) Значение переменной A равно 200 = c8
- 4) Адрес переменной A равен 000000000062FE1C
- 5) Данные по адресу указателя B равны 200 = c8
- 6) Значение указателя B равно 62fe1c = 000000000062FE1C
- 7) Адрес расположения указателя B равен 000000000062FE10



```

1  #include <stdio.h>
2
3  int main()
4  {
5      float x = 1.01, y;
6      float *p;
7      p = &x;
8      y = *p;
9      printf("x = %f, y = %f\n", x, y);
10     (*p)++;
11     printf("x = %f, y = %f\n", x, y);
12     y = 1+*p*y;
13     printf("x = %f, y = %f\n", x, y);
14 }

```

```

x = 1.010000, y = 1.010000
x = 2.010000, y = 1.010000
x = 2.010000, y = 3.030100

```

```

6  void *vp; //нетипизированный указатель
7  float f,*fp;
8  fp = &f;
9  vp = fp; //нетипизированному указателю можно
10         //присвоить указатель любого другого типа
11  fp = (float*)vp; //при обратном присваивании
12                  //необходимо использовать явное преобразование
13                  //нетипизированного указателя

```

3 Арифметика указателей

Арифметические операции: сложение и вычитание

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int *p;
6      int x = 15;
7      p = &x;
8      printf("1) p = %p, &x = %p, *p = %d, x = %d\n", p, &x, *p, x);
9      ++p;
10     printf("2) p = %p, &x = %p, *p = %d, x = %d\n", p, &x, *p, x);
11     p = p + 3;
12     printf("3) p = %p, &x = %p, *p = %d, x = %d\n", p, &x, *p, x);
13     p = p - 4;
14     printf("4) p = %p, &x = %p, *p = %d, x = %d\n", p, &x, *p, x);
15     return 0;
```

```
1) p = 000000000062FE08, &x = 000000000062FE08, *p = 15, x = 15
2) p = 000000000062FE0C, &x = 000000000062FE08, *p = 200, x = 15
3) p = 000000000062FE18, &x = 000000000062FE08, *p = 6487564, x = 15
4) p = 000000000062FE08, &x = 000000000062FE08, *p = 15, x = 15
```

Вычитание указателей и сравнение указателей:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int *p1, *p2;
6      int m[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
7      p1 = &m[0];
8      p2 = &m[7];
9      printf("1) p1 = %p, p2 = %p, *p1=%d, *p2=%d\n", p1, p2, *p1, *p2);
10
11     p1 = p1 + 3;
12     printf("2) p1 = %p, p2 = %p, *p1=%d, *p2=%d\n", p1, p2, *p1, *p2);
13
14     printf("3) p2-p1 = %d\n", p2-p1);
15     printf("4) %d, %d, %d, %d, %d, %d", p1<p2, p1>p2, p1==p2, p1!=p2, p1<=p2, p1>=p2);
16 }
```

```
1) p1 = 000000000062FDE0, p2 = 000000000062FDFC, *p1=0, *p2=7
2) p1 = 000000000062FDEC, p2 = 000000000062FDFC, *p1=3, *p2=7
3) p2-p1 = 4
4) 1, 0, 0, 1, 1, 0
```

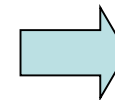
4 Связь указателей и массивов

```
1  #include <stdio.h>
2
3  int main()
4  {
5  int m[5] = {0, 1, 2, 3, 4};
6
7  printf("%p, %p, %p, %p\n", m, &m[0], m+4, &m[4]);
8
9  m[3] = 77;
10 printf("m = %d %d %d %d %d\n", m[0], m[1], m[2], m[3], m[4]);
11
12 *(m+3) = 88;
13 printf("m = %d %d %d %d %d\n", *m, *(m+1), *(m+2), *(m+3), *(m+4));
14 }
```

```
000000000062FE00, 000000000062FE00, 000000000062FE10, 000000000062FE10
m = 0 1 2 77 4
m = 0 1 2 88 4
```


5 Указатели и структуры

```
1  #include <stdio.h>
2  #include <string.h>
3
4  typedef struct
5  {
6      char name[20];
7      int age;
8  } person;
9
10
11 int main()
12 {
13     person P = {"Anna", 20};
14     person *P_pointer; //указатель на структуру
15
16     P_pointer = &P;
17
18     printf("1) %s, %d\n", P.name, P.age);
19     printf("2) %s, %d\n", (*P_pointer).name, (*P_pointer).age);
20     printf("3) %s, %d\n", P_pointer->name, P_pointer->age);
21
22     strcpy(P_pointer->name, "Alex");
23     P_pointer->age = 10;
24     printf("4) %s, %d\n", P.name, P_pointer->age);
25 }
```



```
1) Anna, 20
2) Anna, 20
3) Anna, 20
4) Alex, 10
```

6 Работа с памятью

Подключение библиотеки: `#include <malloc.h>`

Функции: `malloc()`, `calloc()`, `free()`, `_msize()`, `realloc()`

Выделение памяти:

`void* malloc(size_t A);`

`size_t` – это тип `unsigned integer`,

`A` - количество байт, которое надо выделить из памяти,

`void*` - функция возвращает `void`-указатель на выделенную область памяти, его нужно привести к нужному типу.

Пример:

```
int *mas, n = 5;
```

```
mas=(int*) malloc(n*sizeof(int));
```

Выделение памяти для массива, инициализированного нулями :

void* calloc(size_t A, size_t B);

A – количество элементов в массиве,

B – размер в байтах одного элемента,

void* - функция возвращает void-указатель на выделенную область памяти, его нужно привести к нужному типу.

Пример:

```
int *mas1, n = 5;
```

```
char *mas2;
```

```
mas1 = (int*) calloc(n, sizeof(int));
```

```
mas2=(char*) calloc(n*sizeof(char));
```

Перераспределение памяти (изменение размера предварительно выделенной области памяти):

`void* realloc(void* P, size_t N);`

P – указатель на область памяти, размер которой нужно изменить,

N – новый размер области памяти в байтах.

void* - функция возвращает void-указатель на выделенную область памяти, его нужно привести к нужному типу.

Пример:

```
int *mas, n = 5;
```

```
mas=(int*) malloc(n*sizeof(int));
```

```
...
```

```
mas = (int*) realloc(mas, 10 * sizeof(int));
```

Освобождение памяти:

```
void free(void *P);
```

P – указатель на блок памяти, который необходимо освободить

Пример:

```
int *mas, n = 5;  
mas=(int*) malloc(n*sizeof(int));  
...  
free(mas)
```

Размер выделенной области памяти:

```
size_t _msize (void* P);
```

P – указатель на блок памяти

Пример:

```
int *mas, n = 5;  
mas=(int*) malloc(n*sizeof(int));  
printf( "%d\n", _msize(mas) ); // выделенная память  
printf( "%d", _msize(mas)/sizeof(mas[0]) ); //количество элементов
```

```

1  #include <stdio.h>
2  #include <malloc.h>
3
4  int main()
5  {
6      int *mas, n = 5;
7      mas=(int*) malloc(n*sizeof(int));
8      printf("Адрес начала массива: %p\n",mas);
9      printf("Размер массива в байтах: %d\n",_msize(mas));
10     printf("Количество элементов: %d\n", _msize(mas)/sizeof(mas[0]));
11
12     printf("-----\n");
13
14     mas = (int*) realloc(mas, 10 * sizeof(int));
15     printf("Адрес начала массива: %p\n",mas);
16     printf("Размер массива в байтах: %d\n",_msize(mas));
17     printf("Количество элементов: %d\n", _msize(mas)/sizeof(mas[0]));
18
19     free(mas);
20 }

```

```

Адрес начала массива: 0000000000A61400
Размер массива в байтах: 20
Количество элементов: 5
-----
Адрес начала массива: 0000000000A61400
Размер массива в байтах: 40
Количество элементов: 10

```

7 Динамический одномерный массив

```
1 #include <stdio.h>
2 #include <malloc.h>
3
4 int main ()
5 {
6     int *mas; //указатель на массив
7     int n, i;
8
9     printf("Введите размер массива: ");
10    scanf("%d", &n);
11
12    // выделение памяти
13    mas = (int*)malloc(n * sizeof(int));
14
15    // ввод элементов массива
16    for (i = 0; i<n; i++)
17    {
18        printf("mas[%d] = ", i);
19        scanf("%d", &mas[i]);
20    }
21    // вывод элементов
22    printf("Элементы массива: ");
23    for (i = 0; i<n; i++)
24    {
25        printf("%d ", mas[i]);
26    }
27    free(mas);
28 }
```

Введите размер массива: 4
mas[0] = 6
mas[1] = 0
mas[2] = -10
mas[3] = 15
Элементы массива: 6 0 -10 15

```
1 #include <stdio.h>
2 #include <malloc.h>
3
4 int main ()
5 {
6     int *mas; //указатель на массив
7     int n, i;
8
9     printf("Введите размер массива: ");
10    scanf("%d", &n);
11
12    // выделение памяти
13    mas = (int*)malloc(n * sizeof(int));
14
15    // ввод элементов массива
16    for (i = 0; i<n; i++)
17    {
18        printf("mas[%d] = ", i);
19        scanf("%d", mas+i);
20    }
21    // вывод элементов
22    printf("Элементы массива: ");
23    for (i = 0; i<n; i++)
24    {
25        printf("%d ", *(mas+i));
26    }
27    free(mas);
28 }
```

```

1 #include <stdio.h>
2 #include <malloc.h>
3
4 enum menu {add=1, del, show, exit} M = show;
5
6 int main ()
7 {
8     int *mas; //указатель на массив
9     int n = 0; //количество элементов в массиве
10
11     // выделение памяти
12     mas = (int*) malloc(n * sizeof(int));
13
14     printf("МЕНЮ\n");
15     printf("1 - Добавить элемент в конец массива\n");
16     printf("2 - Удалить последний элемент\n");
17     printf("3 - Вывести элементы массива\n");
18     printf("4 - Выход\n");

```

```

МЕНЮ
1 - Добавить элемент в конец массива
2 - Удалить последний элемент
3 - Вывести элементы массива
4 - Выход
Выберите пункт меню: 1
mas[0] = 10
Выберите пункт меню: 1
mas[1] = 20
Выберите пункт меню: 1
mas[2] = 30
Выберите пункт меню: 3
Элементы массива: 10 20 30
Выберите пункт меню: 2
Выберите пункт меню: 3
Элементы массива: 10 20

```

```

19
20 while (M != exit)
21 {
22     printf("Выберите пункт меню: ");
23     scanf("%d", &M);
24
25     switch(M)
26     {
27         case add: n++;
28                 mas = (int*)realloc(mas, n * sizeof(int));
29                 printf("mas[%d] = ", n-1);
30                 scanf("%d", mas+(n-1));
31                 break;
32         case del: n--;
33                 mas = (int*)realloc(mas, n * sizeof(int));
34                 break;
35         case show: printf("Элементы массива: ");
36                    for (int i = 0; i<n; i++)
37                    {
38                        printf("%d ", *(mas+i));
39                    }
40                    printf("\n");
41                    break;
42         case exit: free(mas);
43     };
44 }
45 }

```

```

Выберите пункт меню: 1
mas[2] = 50
Выберите пункт меню: 3
Элементы массива: 10 20 50
Выберите пункт меню: 4

```


Динамическая строка:

```
1  #include <stdio.h>
2  #include <malloc.h>
3  #include <string.h>
4
5  int main()
6  {
7      char *str;
8      str = (char*) malloc(20 * sizeof(char));
9
10     strcpy(str, "Hello, world!");
11
12     printf("%s \n %c \n %d", str, str[4], strlen(str));
13
14     free(str);
15 }
```



```
Hello, world!
o
13
```

8 Динамические матрицы

ПЕРВЫЙ способ (для матрицы NxM):

$$\text{index} = i * M + j;$$

где i - номер текущей строки; j - номер текущего столбца

		j=2		
	0	1	2	3
i=1	4	5	6	7
	8	9	10	11

Индекс выделенного элемента:
 $\text{index} = 1 * 4 + 2 = 6$

```
int *matr; // указатель на матрицу
int N = 3, M = 4;

// выделение памяти под матрицу
matr = (int*) malloc( N*M * sizeof(int) );

// заполнение матрицы значениями

// matr[0][0] = 0;
*(matr + 0*M + 0) = 0; // *(matr) = 0

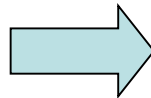
// matr[1][2] = 6;
*(matr + 1*M + 2) = 6;

// matr[2][3] = 11;
*(matr + 2*M + 3) = 11;
```

```

1  #include <stdio.h>
2  #include <malloc.h>
3
4  int main()
5  {
6      int *matr;          //указатель на матрицу
7      int N = 3, M = 4; //размер матрицы
8      int i, j;
9      // выделение памяти под матрицу
10     matr = (int*)malloc( N*M * sizeof(int) );
11
12     // ввод элементов матрицы
13     for (i = 0; i<N; i++) // цикл по строкам
14     {
15         for (j = 0; j<M; j++) // цикл по столбцам
16         {
17             printf("matr[%d][%d] = ", i, j);
18             scanf("%d", ( matr + i*M + j) );
19         }
20     }
21
22     // вывод элементов матрицы
23     for (i = 0; i<N; i++) // цикл по строкам
24     {
25         for (j = 0; j<M; j++) // цикл по столбцам
26         {
27             printf("%5d ", *( matr + i*M + j ));
28         }
29         printf("\n");
30     }
31
32     free(matr);
33 }

```



```

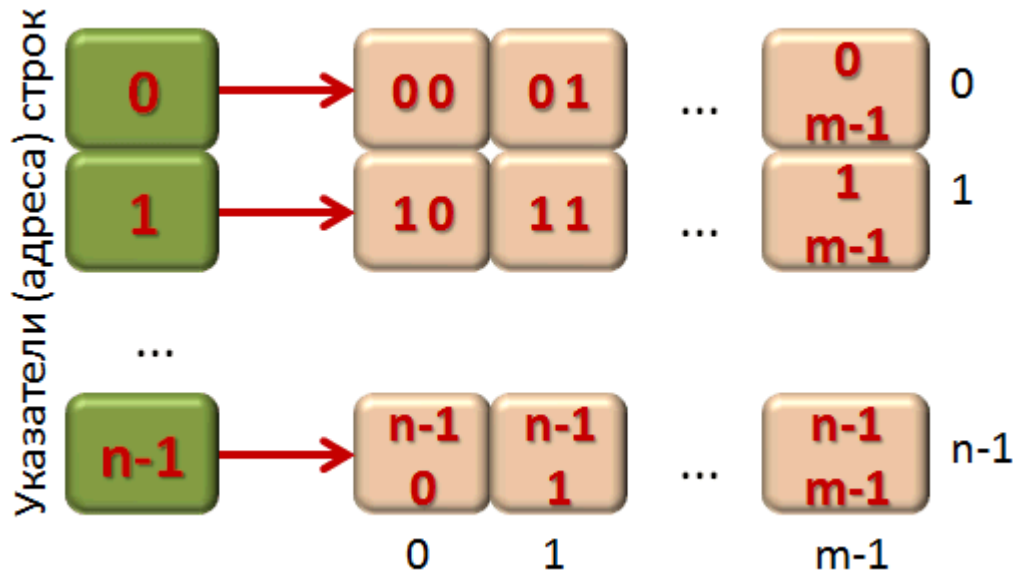
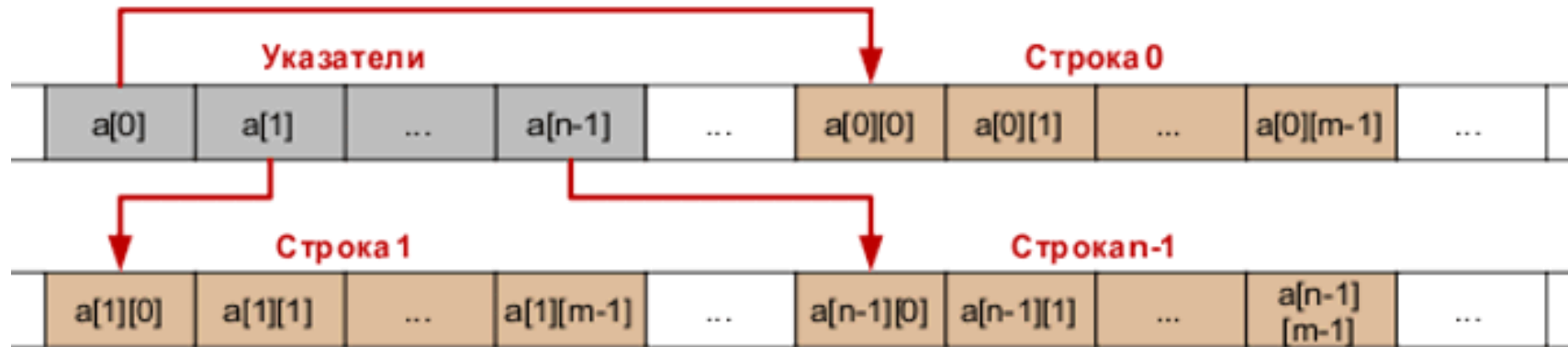
matr[0][0] = 0
matr[0][1] = 1
matr[0][2] = 2
matr[0][3] = 3
matr[1][0] = 4
matr[1][1] = 5
matr[1][2] = 6
matr[1][3] = 7
matr[2][0] = 8
matr[2][1] = 9
matr[2][2] = 10
matr[2][3] = 11

```

	0	1	2	3
0	4	5	6	7
8	9	10	11	

ВТОРОЙ способ (для матрицы $n \times m$) с использованием массива указателей:

`int **a;`



Создание матрицы:

- 1) выделить блок оперативной памяти под массив указателей;
- 2) выделить блоки оперативной памяти под одномерные массивы, представляющие собой строки матрицы;
- 3) записать адреса строк в массив указателей.

Создание матрицы:

1) выделить блок оперативной памяти под массив указателей:

```
int **a; // указатель на указатель
a = (int**)malloc( n * sizeof(int*) ); // выделение памяти под указатели на строки
// (это массив указателей)
```

2) выделить блоки оперативной памяти под одномерные массивы, представляющие собой строки матрицы и записать адреса строк в массив указателей:

```
// ввод элементов матрицы
for (i = 0; i < n; i++) // цикл по строкам
{
    a[i] = (int*) malloc(m * sizeof(int)); // выделение памяти под хранение строк
                                           // и запись адресов строк в массив указателей
    for (j = 0; j < m; j++) // цикл по столбцам
    {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}
```

```
printf("%5d ", a[i][j]);  
printf("%5d ", *(a[i]+j));
```

```
scanf("%d", &a[i][j]);  
scanf("%d", a[i]+j);
```

```
1  #include <stdio.h>  
2  #include <malloc.h>  
3  
4  int main()  
5  {  
6      int **a;  
7      int n = 3, m = 4;  
8      int i, j;  
9  
10     // выделение памяти под указатели на строки матрицы  
11     a = (int**)malloc(n * sizeof(int*));  
12  
13     // ввод элементов матрицы  
14     for (i = 0; i < n; i++) // цикл по строкам  
15     {  
16  
17         // выделение памяти под хранение строк  
18         // и запись адресов строк в массив указателей  
19         a[i] = (int*) malloc(m * sizeof(int));  
20  
21         for (j = 0; j < m; j++) // цикл по столбцам  
22         {  
23             printf("a[%d][%d] = ", i, j);  
24             scanf("%d", &a[i][j]);  
25         }  
26     }
```

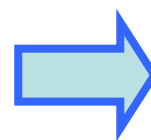
```
27  
28     // вывод элементов матрицы  
29     for (i = 0; i < n; i++) // цикл по строкам  
30     {  
31         for (j = 0; j < m; j++) // цикл по столбцам  
32         {  
33             printf("%5d ", a[i][j]);  
34         }  
35         printf("\n");  
36     }  
37  
38     // очистка памяти  
39     for (i = 0; i < n; i++) // цикл по строкам  
40     {  
41         free(a[i]); // освобождение памяти, выделенной под строку  
42     }  
43     free(a);  
44  
45 }
```

```
a[0][0] = 0  
a[0][1] = 1  
a[0][2] = 2  
a[0][3] = 3  
a[1][0] = 4  
a[1][1] = 5  
a[1][2] = 6  
a[1][3] = 7  
a[2][0] = 8  
a[2][1] = 9  
a[2][2] = 10  
a[2][3] = 11
```

0	1	2	3
4	5	6	7
8	9	10	11

9 Динамический массив структур

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <malloc.h>
4
5  typedef struct
6  {
7      char name[20];
8      int age;
9  } person;
10
11 int main()
12 {
13     person *mas; //указатель на массив структур
14     mas = (person*) malloc(2 * sizeof(person));
15
16     //заполнение 0ого и 1ого элементов массива
17     strcpy(mas[0].name, "Anna"); mas[0].age = 20;
18     strcpy(mas[1].name, "Alex"); mas[1].age = 10;
19
20     //выделение памяти ещё для одного человека
21     mas = (person*) realloc (mas, 3*sizeof(person));
22     strcpy( (mas+2)->name, "Rich"); (mas+2)->age = 30;
23
24     // поиск информации о людях с возрастом более 15 лет
25     for (int i=0; i<3; i++)
26     {
27         if (mas[i].age >15)
28         {
29             printf("%s, %d\n", mas[i].name, (mas+i)->age);
30         }
31     }
32     free(mas);
33 }
```



Anna, 20
Rich, 30